Chapter 9 — Retrieval-Augmented Generation (RAG)

9.1 RAG Architecture and When to Use It

Conceptual Deep Dive

RAG = retrieve relevant passages at query time, place them in the prompt, and instruct the model to answer *only* from them with citations. It addresses stale knowledge, private data, and hallucination, and keeps knowledge updatable without retraining. Pipeline: **ingest** (parse → clean → chunk → embed → index) and **query** (rewrite → retrieve → re-rank → assemble context → generate → verify). Choose: prompt-only for stable general knowledge; RAG for large/changing/private corpora; long-context stuffing for small corpora if cost allows; fine-tuning to change *behaviour/style*, not to inject facts; tools/SQL for structured data.

Cross-Connection Scenarios

Embeddings and indexes (8), structured citations (7), eval triad (10), agentic RAG (11), ACL security (14).

Production Anti-Patterns

- Fine-tuning to "teach" the model documents instead of retrieving them.
- No "I don't know" path when retrieval is empty or weak.
- Evaluating only the final answer, never retrieval separately.

Minimal Working Code

```
def answer(question, retriever, llm, min_score=0.3, k=5):
    hits = [h for h in retriever(question, k) if h["score"] >= min_score]
    if not hits:
        return {"answer": "I don't have enough information.", "sources": []
}
    ctx = "\n\n".join(f"[{i+1}] {h['text']}" for i, h in enumerate(hits))
    prompt = (f"Answer using only the sources. Cite as [n]. If unsupported, say so.\n\n"
              f"Sources:\n{ctx}\n\nQuestion: {question}")
    return {"answer": llm(prompt), "sources": [h["id"] for h in hits]}
```

9.2 Ingestion and Chunking Strategies

Conceptual Deep Dive

Parsing quality bounds everything: use layout-aware parsers for PDFs (tables, columns), OCR for scans, and preserve headings. Chunking trade-off: small chunks → precise retrieval but lost context; large → context-rich but noisy and costly. Strategies: **fixed-size with overlap** (baseline, 200–500 tokens, 10–20% overlap); **recursive character splitting** (paragraph → sentence → word); **structure-aware** (Markdown headings, HTML sections, code by AST/function); **semantic chunking** (split where adjacent-sentence embedding similarity drops); **sentence-window** (retrieve a sentence, return neighbours); **parent-document / small-to-big** (index small chunks, return the parent section); **contextual retrieval** (prepend an LLM-generated situating summary to each chunk before embedding/BM25); **proposition indexing**; **RAPTOR** (recursive summary tree). Attach metadata: source, section path, page, date, ACL, version. Deduplicate and handle updates by stable chunk ids.

Cross-Connection Scenarios

Token counting (5.3), embedding versioning (8.6), ACL metadata (14), chunk-level evals (10).

Production Anti-Patterns

- Splitting mid-table or mid-code-block.
- Dropping headings so chunks lose their topic.
- Re-ingesting everything on small document edits (no incremental updates).

Minimal Working Code

```
import re

def recursive_split(text: str, max_chars: int = 1200, overlap: int = 150,
                    seps=("\n\n", "\n", ". ", " ")) -> list[st
r]:
    if len(text) <= max_chars:
        return [text]
    for sep in seps:
        parts = text.split(sep)
        if len(parts) > 1:
            chunks, cur = [], ""
            for p in parts:
                if len(cur) + len(p) + len(sep) <= max_chars:
                    cur += (sep if cur else "") + p
                else:
                    if cur:
                        chunks.append(cur)
                    cur = (cur[-overlap:] + sep if cur and overlap else "") + p
            if cur:
                chunks.append(cur)
            return [c for ch in chunks for c in (recursive_split(ch, max_chars, overlap, seps)
                                                 if len(ch) > max_chars else [ch])]
    return [text[i:i + max_chars] for i in range(0, len(text), max_chars - overlap)]

def markdown_sections(md: str):
    path, buf = [], []
    for line in md.splitlines():
        if m := re.match(r"(#+)\s+(.*)", line):
            if buf:
                yield " > ".join(path), "\n".join(buf)
                buf = []
            path = path[:len(m.group(1)) - 1] + [m.group(2)]
        else:
            buf.append(line)
    if buf:
        yield " > ".join(path), "\n".join(buf)
```

9.3 Advanced Retrieval: Query Transformation, HyDE, Fusion, GraphRAG

Conceptual Deep Dive

- **Query rewriting**: resolve pronouns from chat history, expand abbreviations, fix typos.
- **Multi-query**: generate N paraphrases, retrieve for each, fuse.
- **HyDE** (Hypothetical Document Embeddings): have the LLM write a fake answer, embed *that*; answers resemble documents more than questions do.
- **Step-back prompting**: retrieve for a more general question too.
- **Decomposition**: split multi-hop questions into sub-queries.
- **Parent-document retrieval**: match on child chunks, return the parent for context.
- **Reciprocal Rank Fusion** (see 8.5) merges dense, sparse and multi-query result lists.
- **Metadata self-query**: LLM turns "docs from 2024 about billing" into a filter + query.

- **GraphRAG**: extract entities/relations into a knowledge graph, detect communities, summarise them; excels at global "what are the main themes" questions and multi-hop reasoning, at high ingestion cost. Lighter variants: LightRAG, LazyGraphRAG, entity-linked chunk graphs.
- **Agentic / Self-RAG / CRAG**: the model decides whether to retrieve, grades retrieved docs, and re-queries or falls back to web search.

Cross-Connection Scenarios

Agents decide retrieval dynamically (11); latency budget limits how many LLM calls precede generation (2.4); evals compare each technique (10).

Production Anti-Patterns

- Stacking every technique "just in case" (latency, cost, compounding errors).
- HyDE on domains where the LLM hallucinates plausible-but-wrong jargon.
- Building a knowledge graph when plain hybrid retrieval already hits target recall.

Minimal Working Code

```
import asyncio
from collections import defaultdict

def rrf(lists, k=60):
    s = defaultdict(float)
    for L in lists:
        for r, d in enumerate(L, 1):
            s[d] += 1 / (k + r)
    return sorted(s, key=s.get, reverse=True)

async def hyde_multi_query(question, llm, search, n=3):
    fake = await llm(f"Write a short passage that answers: {question}")
    paraphrases = (await llm(f"Give {n} paraphrases, one per line: {question}")).splitlines()
    queries = [question, fake] + paraphrases[:n]
    results = await asyncio.gather(*(search(q) for q in queries))
    return rrf(results)[:10]
```

9.4 Re-ranking

Conceptual Deep Dive

First-stage retrieval (bi-encoder) is fast but coarse: query and document are embedded independently. A **cross-encoder** reads query and document *together* and outputs a relevance score — far more accurate, but O(candidates) model calls, so apply it to the top 50–200 only. Options: hosted (**Cohere Rerank**, Voyage), open (**BGE-reranker**, mxbai, Jina), late-interaction (ColBERT), or LLM-as-reranker (listwise). Typical pattern: retrieve 100 → rerank → keep top 5–10. Re-ranking also lets you use compressed/cheap first-stage indexes.

Cross-Connection Scenarios

Compensates for quantised embeddings (8.2); latency tuning (13); measured via nDCG/MRR (10).

Production Anti-Patterns

- Reranking thousands of candidates (latency explosion).
- Truncating documents beyond the reranker's max length without noticing.
- Dropping the first-stage score entirely when the reranker is out-of-domain.

Minimal Working Code

```
# pip install sentence-transformers
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("BAAI/bge-reranker-base")

def rerank(query: str, docs: list[str], top_n: int = 5):
    scores = reranker.predict([(query, d) for d in docs])
    order = sorted(range(len(docs)), key=lambda i: -scores[i])[:top_n]
    return [(docs[i], float(scores[i])) for i in order]
```

9.5 Context Assembly, Citations and Hallucination Mitigation

Conceptual Deep Dive

Assembly: order by relevance (best first and last), dedupe, compress (extractive or LLM summarisation), include source ids, and cap tokens. Hallucination mitigation stack: (1) grounded prompt ("only from sources; say unknown"), (2) retrieval thresholds and abstention, (3) **citation enforcement** with quote extraction, (4) **verification**: a second pass checks each claim is entailed by cited text (NLI or LLM judge), (5) lower temperature, (6) structured output with answerable: bool, (7) human fallback for high-stakes domains. Distinguish *intrinsic* hallucination (contradicts the source) from *extrinsic* (unsupported by it) and measure both (Chapter 10 *faithfulness*).

Cross-Connection Scenarios

Structured output (7) carries citations; judge models (10); guardrails (14).

Production Anti-Patterns

- Citations that are decorative (id not actually supporting the claim).
- Giving the model 30 passages and expecting equal attention.
- Showing confident answers when top retrieval score is poor.

Minimal Working Code

```
import re

def verify_citations(answer: str, sources: dict[int, str]) -> list[str]:
    problems = []
    for sentence in re.split(r"(?<=[.!?])\s+", answer):
        cites = [int(c) for c in re.findall(r"\[(\d+)\]", sentence)]
        if not cites:
            problems.append(f"uncited: {sentence}")
        for c in cites:
            if c not in sources:
                problems.append(f"bad citation [{c}]")
    return problems

print(verify_citations("Paris is the capital [1]. It rains.", {1: "Paris is the capital of
 France."}))
```

9.6 RAG Over Structured Data, Multimodal and Freshness

Conceptual Deep Dive

Text-to-SQL over warehouses: supply schema + sample rows, restrict to read-only credentials, validate SQL (parse, allow-list tables, add LIMIT), execute, and summarise. **Multimodal RAG**: embed images/pages with models like CLIP/ColPali, or caption into text. **Freshness**: incremental ingestion via change-data-capture, TTLs, and recency-boosted scoring. **Caching** retrieved results for hot queries. **Conversation-aware RAG** rewrites each turn into a standalone query.

Cross-Connection Scenarios

Tool use (7), security (14), serving cache (13).

Production Anti-Patterns

- Giving text-to-SQL write access or unscoped schema.
- Stale index with no freshness metric.
- Embedding screenshots of tables as plain OCR text and losing structure.

Minimal Working Code

```
import re

ALLOWED_TABLES = {"orders", "customers"}

def safe_sql(sql: str) -> str:
    s = sql.strip().rstrip(";")
    if not re.match(r"(?is)^select\b", s) or re.search(r"(?i)\b(insert|update|delete|drop|
alter|;)\b", s):
        raise ValueError("only single SELECT allowed")
    tables = set(re.findall(r"(?i)\b(?:from|join)\s+(\w+)", s))
    if not tables <= ALLOWED_TABLES:
        raise ValueError(f"table not allowed: {tables - ALLOWED_TABLES}")
    return s if re.search(r"(?i)\blimit\b", s) else s + " LIMIT 200"

print(safe_sql("select * from orders"))
```

9.7 Diagnosing a Failing RAG System

Conceptual Deep Dive

Debug in pipeline order and fix the earliest broken stage. (1) **Is the answer in the corpus?** If not, it is an ingestion/coverage problem. (2) **Was it parsed and chunked intact?** Inspect the actual chunks. (3) **Is it retrieved at all (recall@50)?** If not: embedding model, query rewriting, hybrid search, metadata filters. (4) **Is it ranked in the top-k you pass (recall@k)?** If not: reranker, k, fusion. (5) **Does the prompt present it clearly?** Ordering, truncation, noise. (6) **Does the model use it faithfully?** Prompt, model choice, citations. Build a per-stage dashboard from labelled queries. Edge cases: abbreviations and synonyms absent from the corpus, queries needing aggregation across many documents (retrieval is the wrong tool; use SQL or map-reduce), time-sensitive answers choosing outdated documents, and contradictory sources.

Cross-Connection Scenarios

Metrics (10.2), failure taxonomy (2.5), observability traces (14.1).

Production Anti-Patterns

- Swapping the LLM before checking whether the evidence was retrieved.
- Increasing top-k to 30 to "fix" recall, diluting precision.
- No handling of conflicting documents (prefer newest/authoritative and say so).

Minimal Working Code

```
def stage_report(cases, retrieve, rerank, k_final=5):
    """cases: [{'q': str, 'gold': set[str]}]; retrieve(q)->list[id]; rerank(q, ids)->
;list[id]."""
    r_any = r_final = 0
    for c in cases:
        cands = retrieve(c["q"])
        final = rerank(c["q"], cands)[:k_final]
```

```
        r_any += bool(c["gold"] & set(cands))
        r_final += bool(c["gold"] & set(final))
    n = len(cases)
    return {"recall@candidates": r_any / n, "recall@final": r_final / n,
            "reranker_loss": (r_any - r_final) / n}

print(stage_report([{"q": "x", "gold": {"d1"}}], lambda q: ["
;d2", "d1"], lambda q, ids: ids[::-1]))
```

```
from datetime import date

def prefer_latest(hits: list[dict]) -> list[dict]:
    """Among chunks of the same document family, keep only the newest version.""
"
    best = {}
    for h in hits:
        fam = h["family"]
        if fam not in best or h["effective"] > best[fam]["effective"]:
            best[fam] = h
    return sorted(best.values(), key=lambda h: -h["score"])

print(prefer_latest([{"family": "policy", "effective": date(2024, 1, 1), &q
uot;score": .9},
                     {"family": "policy", "effective": date(2026, 1, 1), &q
uot;score": .7}]))
```

9.8 Conversational RAG, Personalisation and Caching

Conceptual Deep Dive

In multi-turn chat, rewrite each user message into a **standalone query** using recent history ("what about the premium plan?" becomes "price of the premium plan"). Decide per turn whether retrieval is needed (small talk, follow-ups answerable from prior context) with a cheap router. Personalise with user attributes (region, plan, role) as retrieval filters and prompt context, never as hidden ranking that the user cannot inspect. Cache at three levels: query-embedding cache, retrieval-result cache keyed by (normalised query, filters, index version), and answer cache for FAQ-like queries with invalidation on index updates. Edge cases: topic switches that make the history rewrite pull in stale context, cache keys that omit the ACL (cross-user leakage), and coreference to previous answers rather than previous questions.

Cross-Connection Scenarios

Memory (11.4), semantic caching (4.6, 13.6), ACL filters (8.5, 14).

Production Anti-Patterns

- Concatenating the whole chat history into the retrieval query.
- Caching answers without the user's permission scope in the key.
- Never invalidating caches after re-indexing.

Minimal Working Code

```
def standalone_query(llm, history: list[tuple[str, str]], question: st
r, window: int = 4) -> str:
    recent = "\n".join(f"{r}: {t}" for r, t in history[-window:])
    return llm(f"Rewrite the final user question so it is understandable without the conversation. &
quot;
               f"Keep all entities. Output only the question.\n\n{recent}\nuser: {question}")

fake = lambda p: "What is the price of the premium plan?"
print(standalone_query(fake, [("user", "Tell me about plans"), ("assistant"
, "Basic and premium...")],
```

```
                  "and the premium one?"))
```

```
import hashlib
import json

def cache_key(query: str, filters: dict, index_version: str, acl_groups: list[str]) -> str:
    blob = json.dumps({"q": " ".join(query.lower().split()), "f": filters,
                       "v": index_version, "acl": sorted(acl_groups)}, sort_keys=True
)
    return hashlib.sha256(blob.encode()).hexdigest()

print(cache_key("Refund policy", {"region": "EU"}, "idx-2026-10-22&quo
t;, ["staff"]))
```